

COMP64702: Transforming Text Into Meaning

Language Modelling

Chenghua Lin

E: `chenghua.lin@manchester.ac.uk`

X: `@chenghua.lin`

Department of Computer Science
The University of Manchester

In previous lecture...

- We studied **Vector Representations of Text**

In this lecture...

Our first NLP problem: Language Modelling

What is the probability of a given sequence of words in a particular language (e.g. English)?

In this lecture...

Our first NLP problem: Language Modelling

What is the probability of a given sequence of words in a particular language (e.g. English)?

Odd problem. Applications?

Applications of LMs

- Word likelihood for query completion in information retrieval
(“Is Manchester” → try it on your search engine)

Applications of LMs

- Word likelihood for query completion in information retrieval
(“Is Manchester” → try it on your search engine)

Applications of LMs

- Word likelihood for query completion in information retrieval (“Is Manchester” → try it on your search engine)
- Grammatical error detection (“You’re welcome” or “Your welcome”?)

Applications of LMs

- Word likelihood for query completion in information retrieval (“Is Manchester” → try it on your search engine)
- Grammatical error detection (“You’re welcome” or “Your welcome”?)
- Speech recognition (“I scream is delicious.” or “Ice cream is delicious.”?)

Traditional AI approach

- Look for interpretation which is
 - grammatical
 - semantically sensible
 - pragmatically plausible in context
- Build explicit models of rules describing how language works
- Poor performance in real world
 - hard to maintain wide-coverage systems
 - too much knowledge needed for semantics

Statistical Approaches

- No explicit models of grammar, meaning, context, etc.
- Instead, collect statistics such as
 - how often different words appear
 - how often pairs (or triples, etc.) of words appear next to each other
 - Or other more complex statistical patterns
- Use this statistics to perform NLP tasks
 - speech, text prediction, translation, ...

N-gram Statistics

- See how often word(s) occur
 - single words (unigram)
 - word pairs (bigram)
 - word triples (trigram)
- Assign a probability to each sequence interpretation, using the statistics
- Pick highest-probability one

Language Models

- **N -gram models**: probabilistic models that predict the next word from the previous $N - 1$ words
- Such statistical models of word sequences are also called **language models** (LMs).

Problem Setup

Training data is a (often large) set of sentences $\mathbf{x}^m$ with words x_n :

$$D_{train} = \{\mathbf{x}^1, \dots, \mathbf{x}^M\}$$
$$\mathbf{x} = [x_1, \dots, x_N]$$

for example:

$$\mathbf{x} = [<s>, \text{The}, \text{water}, \text{is}, \text{clear}, ., </s>]$$

$<s>$: denotes start of the sentence

$</s>$: denotes end of the sentence

Corpus

- Obtain statistics from a *corpus*
 - representative collection of documents
- British National Corpus (BNC) ↓
 - 100M words in 4K documents
 - newspaper articles, academic papers, school essays, memos, radio phone-ins, etc.
 - <http://www.natcorp.ox.ac.uk/>
- Google n-gram web corpus ↓
 - **1,024,908,267,229** words

Calculate sentence probabilities

We want to learn a model that returns the **probability of an unseen sentence $\mathbf{x}$** :

$$P(\mathbf{x}) = P(x_1, \dots, x_n), \text{ for } \forall \mathbf{x} \in V^{\text{max_}N}$$

V is the vocabulary and $V^{\text{max_}N}$ all possible sentences.

Calculate sentence probabilities

We want to learn a model that returns the **probability of an unseen sentence $\mathbf{x}$** :

$$P(\mathbf{x}) = P(x_1, \dots, x_n), \text{ for } \forall \mathbf{x} \in V^{\text{max_}N}$$

V is the vocabulary and $V^{\text{max_}N}$ all possible sentences.

How to compute probability?

Unigram language model

Multiply the probability of each word appearing in the sentence $\mathbf{x}$ computed over the entire corpus:

$$P(\mathbf{x}) = \prod_{n=1}^N P(x_n) = \prod_{n=1}^N \frac{c(x_n)}{\sum_{x \in V} c(x)}$$

Unigram language model

Multiply the probability of each word appearing in the sentence $\mathbf{x}$ computed over the entire corpus:

$$P(\mathbf{x}) = \prod_{n=1}^N P(x_n) = \prod_{n=1}^N \frac{c(x_n)}{\sum_{x \in V} c(x)}$$

<s> i love playing basketball </s>

<s> arctic monkeys are from Manchester </s>

<s> i study in Manchester uni </s>

Unigram language model

Multiply the probability of each word appearing in the sentence $\mathbf{x}$ computed over the entire corpus:

$$P(\mathbf{x}) = \prod_{n=1}^N P(x_n) = \prod_{n=1}^N \frac{c(x_n)}{\sum_{x \in V} c(x)}$$

<s> i love playing basketball </s>

<s> arctic monkeys are from Manchester </s>

<s> i study in Manchester uni </s>

$$P(\text{i love}) = P(\text{i})P(\text{love}) = \frac{2}{20} \cdot \frac{1}{20} = 0.005 \quad \downarrow$$

What could go wrong?

<s> i love playing basketball </s>

<s> arctic monkeys are from Manchester </s>

<s> i study in Manchester uni </s>

- The most probable word is <s> ($\frac{3}{20}$)

What could go wrong?

<s> i love playing basketball </s>

<s> arctic monkeys are from Manchester </s>

<s> i study in Manchester uni </s>

- The most probable word is <s> ($\frac{3}{20}$)
- The most probable single-word sentence is “<s>”

What could go wrong?

<s> i love playing basketball </s>

<s> arctic monkeys are from Manchester </s>

<s> i study in Manchester uni </s>

- The most probable word is <s> ($\frac{3}{20}$)
- The most probable single-word sentence is “<s>”
- The most probable two-word sentence is “<s> <s>”

What could go wrong?

<s> i love playing basketball </s>

<s> arctic monkeys are from Manchester </s>

<s> i study in Manchester uni </s>

- The most probable word is <s> ($\frac{3}{20}$)
- The most probable single-word sentence is "<s>"
- The most probable two-word sentence is "<s> <s>"
- The most probable N -word sentence is $N \times$ "<s>"

Maximum Likelihood Estimation

Instead of assuming **independence**:

$$P(\mathbf{x}) = \prod_{n=1}^N P(x_n)$$

We assume that each word is **dependent** on all previous ones:

$$\begin{aligned} P(\mathbf{x}) &= P(x_1, \dots, x_N) \\ &= P(x_1)P(x_2 \dots x_N | x_1) \\ &= P(x_1)P(x_2 | x_1) \dots P(x_N | x_1, \dots, x_{N-1}) \\ &= \prod_{n=1}^N P(x_n | x_1, \dots, x_{n-1}) \quad (\text{chain rule}) \quad \checkmark \end{aligned}$$

Maximum Likelihood Estimation

Instead of assuming **independence**:

$$P(\mathbf{x}) = \prod_{n=1}^N P(x_n)$$

We assume that each word is **dependent** on all previous ones:

$$\begin{aligned} P(\mathbf{x}) &= P(x_1, \dots, x_N) \\ &= P(x_1)P(x_2 \dots x_N | x_1) \\ &= P(x_1)P(x_2 | x_1) \dots P(x_N | x_1, \dots, x_{N-1}) \\ &= \prod_{n=1}^N P(x_n | x_1, \dots, x_{n-1}) \quad (\text{chain rule}) \end{aligned}$$

What could go wrong?

Problems with MLE

Let's analyse this:

$$P(\mathbf{x}) = P(x_1)P(x_2|x_1)P(x_3|x_2, x_1)\dots P(x_N|x_1, \dots, x_{N-1})$$

$$P(x_n|x_{n-1}\dots x_1) = \frac{c(x_1\dots x_{n-1}, x_n)}{c(x_1\dots x_{n-1})}$$

As we condition on more words, the counts become **sparser**

Bigram Language Models

Assume that the choice of a word **depends only on the one before it**:

$$P(\mathbf{x}) = \prod_{n=1}^N P(x_n | x_{n-1}) = \prod_{n=1}^N \frac{c(x_{n-1}, x_n)}{c(x_{n-1})} \quad \checkmark$$

k-th order **Markov assumption**:

$$P(x_n | x_{n-1}, \dots, x_1) \approx P(x_n | x_{n-1}, \dots, x_{n-k}) \quad |$$

with $k=1$



Bigram Language Models

Assume that the choice of a word **depends only on the one before it**:

$$\begin{aligned} P(\mathbf{x}) &= \prod_{n=1}^N P(x_n | x_{n-1}) = \prod_{n=1}^N \frac{P(x_{n-1}, x_n)}{P(x_{n-1})} \\ &= \prod_{n=1}^N \frac{c(x_{n-1}, x_n)}{c(x_{n-1})} \end{aligned}$$

k-th order **Markov assumption**:

$$P(x_n | x_{n-1}, \dots, x_1) \approx P(x_n | x_{n-1}, \dots, x_{n-k})$$

with $k=1$



Bigram LM: From counts to probabilities

Unigram counts:

arctic	monkeys	are	my	favourite	band
100	600	4000	3000	500	200

Bigram counts (rows: x_{i-1} , cols: x_i):

	arctic	monkeys	are	my	favourite	band
arctic	0	10	2	0	0	0
monkeys	0	0	250	1	5	0
are	3	45	0	600	25	1
my	0	2	0	1	300	5
favourite	0	1	0	0	0	50
band	0	0	3	10	0	0

Bigram LM: From counts to probabilities

From the bigram count matrix, compute probabilities by dividing each cell by the appropriate unigram count for its row.

Bigram probabilities (rows: x_{i-1} , cols: x_i):

	arctic	monkeys	are	my	favourite	band
arctic	0	0.1	0.02	0	0	0
monkeys	0	0	0.417	0.0017	0.008	0
are	0.0008	0.0113	0	0.15	0.0063	0.00003
my	0	0.0007	0	0.0003	0.1	0.0017
favourite	0	0.002	0	0	0	0.1
band	0	0	0.015	0.05	0	0

Bigram LM: From counts to probabilities

Unigram counts:

arctic	monkeys	are	my	favourite	band
100	600	4000	3000	500	200

Bigram counts (rows: x_{i-1} , cols: x_i):

	arctic	monkeys	are	my	favourite	band
arctic	0	10	2	0	0	0
monkeys	0	0	250	1	5	0
are	3	45	0	600	25	1
my	0	2	0	1	300	5
favourite	0	1	0	0	0	50
band	0	0	3	10	0	0

$\mathbf{x} = [\text{arctic}, \text{monkeys}, \text{are}, \text{my}, \text{favourite}, \text{band}]$

What is $P(\mathbf{x})$?

Example: Bigram language model

$\mathbf{x} = [\text{arctic}, \text{monkeys}, \text{are}, \text{my}, \text{favourite}, \text{band}]$

$$\begin{aligned} P(\mathbf{x}) &= P(\text{monkeys}|\text{arctic})P(\text{are}|\text{monkeys})P(\text{my}|\text{are}) \\ &\quad P(\text{favourite}|\text{my})P(\text{band}|\text{favourite}) \\ &= \frac{c(\text{arctic}, \text{monkeys})}{c(\text{arctic})} \cdots \frac{c(\text{favourite}, \text{band})}{c(\text{favourite})} \\ &= 0.1 \cdot 0.417 \cdot 0.15 \cdot 0.1 \cdot 0.1 \\ &= 0.00006255 \end{aligned}$$

Example: Bigram/Trigram Models

<s> a blue house </s>

<s> a grey house </s>

<s> the grey house has the blue table </s>

- Probabilities of some bigrams:

- $P(a|<s>) = \frac{2}{3}$; $P(the|<s>) = \frac{1}{3}$; $P(</s>|house) = \frac{2}{3}$;
 $P(house|grey) = \frac{2}{2} = 1$

- Probabilities of some tri-grams:

- $P(blue|<s> a) = \frac{1}{2}$; $P(house|a blue) = \frac{1}{1}$

Longer contexts (N-gram LMs)

$$P(x|context) = \frac{P(context, x)}{P(context)} = \frac{c(context, x)}{c(context)}$$

- In bigram LM *context* is x_{n-1} , trigram x_{n-2}, x_{n-1} , etc.
- The longer the context:
 - the more likely to capture long-range dependencies:
“*I saw a tiger that was really very ...*” fierce or talkative?
 - the sparser the counts (zero probabilities)
- 5-grams and training sets with billions of tokens are common.

Generating sentences using a Unigram Model

- Random sentences with words selected according to their unigram probabilities (based on a corpus of airline reservation queries):
 - Of aircraft fare east
 - The the stopover city the coach the one_way frequent in travelling flights
 - Two on flight really Boston me like
 - And like six would these fifty

Generating sentences using a bigram Model

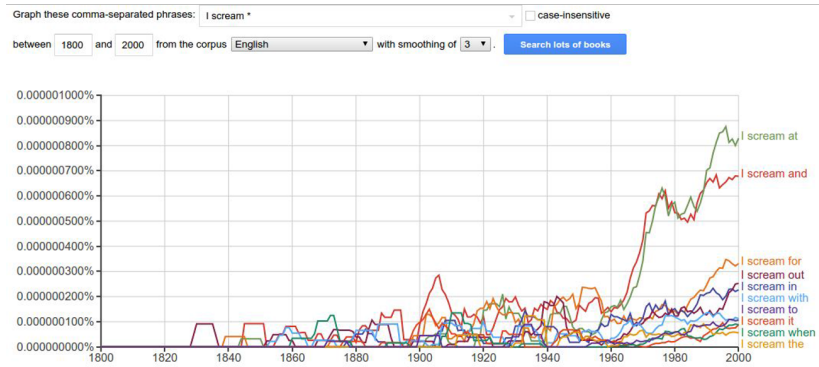
- Random sentences using bigram probabilities:
 - I want a first class
 - I would like to San_Francisco
 - Are the next week on AA eleven ten
 - In Denver on October
 - What is the city of three pm
 - Two on flight really Boston me like
 - And like six would these fifty

Generating sentences using a Trigram Model

- Random sentences using trigram probabilities:
 - How much does it cost
 - I'd like to depart before five o'clock pm
 - How many stops does Delta flight five eleven o'clock pm that go from what am
 - I need to Philadelphia
 - Which flight do these flights leave after four pm and lunch and
 - Two on flight really Boston me like
 - And like six would these fifty

Google Books n -gram Viewer

- Google Books n -grams
- <https://books.google.com/ngrams> ✓



Unknown Words

- If a word was never encountered in training, any sentence containing it will have probability 0
- It happens:
 - all corpora are finite
 - new words emerging

Unknown Words

- If a word was never encountered in training, any sentence containing it will have probability 0
- It happens:
 - all corpora are finite
 - new words emerging
- Common solutions:
 - use a specific UNKNOWN to manage all the words not in the vocabulary
 - Use classes of unknown words, e.g. names and numbers

Evaluation

How do we train and evaluate our language models?

- We need train/dev/test data
- Evaluation approaches

Intrinsic Evaluation: Accuracy

- How well does our LM predict the next word?
- **I always order pizza with cheese and...**
 - mushrooms?
 - bread?
 - and?
- Accuracy: how often the LM predicts the correct word
- The higher the better

Intrinsic Evaluation: Perplexity

- **Perplexity:** measures the ability of the model to predict word of an unseen text
 - Unseen = not in training data, e.g., test set
 - The average negative log-likelihood of the test set $\mathbf{x} = [x_1, \dots, x_N]$, normalised by the number of words N .
 - The *lower* the better. $\checkmark$ (low mean less surprise)

$$\text{Perplexity}(w) = \exp \left(-\frac{\sum_{d=1}^D \log P(w_d)}{\sum_{d=1}^D N_d} \right) \quad \downarrow$$

if $p(w_d)$ is high (1), you are sure what to predict.

if $p(w_d)$ is 0 or very low, you are unlikely to know what to predict

Extrinsic Evaluation

(downstream task first to do evaluation, like below)

- Sentence completion
- Grammatical error correction: detecting “odd” sentences and propose alternatives
- Natural language generation: prefer more “natural” sentences
- Speech recognition
- Machine translation

Smoothing

- What happens when words that are in our vocabulary appear with an unseen context in test data?

Smoothing

- What happens when words that are in our vocabulary appear with an unseen context in test data?
- They will be assigned with zero probability

Smoothing

- What happens when words that are in our vocabulary appear with an unseen context in test data?
- They will be assigned with zero probability

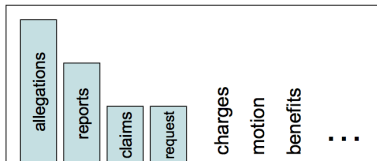


Smoothing (or discounting) to the rescue: Steal from the rich and give to the poor!

Smoothing intuition

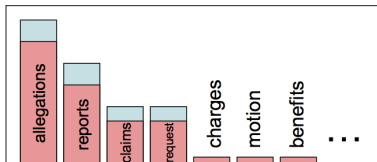
- We often want to make estimates from sparse statistics:

$P(w \mid \text{denied the})$
3 allegations
2 reports
1 claims
1 request
7 total



- Smoothing flattens spiky distributions so they generalize better

$P(w \mid \text{denied the})$
2.5 allegations
1.5 reports
0.5 claims
0.5 request
2 other
7 total



Taking from the frequent and giving to the rare (discounting)

Add-1 Smoothing

Add-1 (or Laplace) smoothing adds one to all counts:

$$P_{Add-1}(x_n) = \frac{c(x_n) + 1}{\text{Corpus size} + |V|}$$

Pretend we have seen all words at least once!

Add-k Smoothing

Add-1 puts too much probability mass to unseen words, better to add- k , $k < 1$:

$$P_{Add-k}(x_n) = \frac{counts(x_n) + k}{corpus\ size + k|V|}$$

Add-k Smoothing

Add-1 puts too much probability mass to unseen words, better to add- k , $k < 1$:

$$P_{Add-k}(x_n) = \frac{counts(x_n) + k}{corpus\ size + k|V|}$$

k is a hyperparameter: choose optimal value on the dev set!

Interpolation

Longer contexts are more informative:

dog bites ... better than bites ...

but only if they are frequent enough:

canid bites ...better than bites ...?

Interpolation

Longer contexts are more informative:

dog bites ... better than bites ...

but only if they are frequent enough:

canid bites ...better than bites ...?

Can we combine evidence from unigram, bigram and trigram probabilities?

Simple Linear Interpolation

For a trigram LM:

$$\begin{aligned} P_{SLI}(x_n|x_{n-1}, x_{n-2}) = & \lambda_3 P(x_n|x_{n-1}, x_{n-2}) \\ & + \lambda_2 P(x_n|x_{n-1}) \\ & + \lambda_1 P(x_n) \quad \lambda_i > 0, \sum \lambda_i = 1 \end{aligned}$$

Simple Linear Interpolation

For a trigram LM:

$$\begin{aligned} P_{SLI}(x_n|x_{n-1}, x_{n-2}) &= \lambda_3 P(x_n|x_{n-1}, x_{n-2}) \\ &\quad + \lambda_2 P(x_n|x_{n-1}) \\ &\quad + \lambda_1 P(x_n) \quad \lambda_i > 0, \sum \lambda_i = 1 \end{aligned}$$

- Weighted average of unigram, bigram and trigram probabilities

Simple Linear Interpolation

For a trigram LM:

$$\begin{aligned}P_{SLI}(x_n|x_{n-1}, x_{n-2}) &= \lambda_3 P(x_n|x_{n-1}, x_{n-2}) \\&\quad + \lambda_2 P(x_n|x_{n-1}) \\&\quad + \lambda_1 P(x_n) \quad \lambda_i > 0, \sum \lambda_i = 1\end{aligned}$$

- Weighted average of unigram, bigram and trigram probabilities
- How we choose the value of λ s?

Simple Linear Interpolation

For a trigram LM:

$$\begin{aligned} P_{SLI}(x_n|x_{n-1}, x_{n-2}) = & \lambda_3 P(x_n|x_{n-1}, x_{n-2}) \\ & + \lambda_2 P(x_n|x_{n-1}) \\ & + \lambda_1 P(x_n) \quad \lambda_i > 0, \sum \lambda_i = 1 \end{aligned}$$

- Weighted average of unigram, bigram and trigram probabilities
- How we choose the value of λ s? Parameter tuning on the dev set!

Higher-order models are more informative and less robust, while lower-order are less informative but more robust.

Backoff

Start with n-gram order of k but if the counts are 0 use $k - 1$:

$$BO(x_n | x_{n-1} \dots x_{n-k}) = \begin{cases} P(x_n | x_{n-1} \dots x_{n-k}), & \text{if } c(x_n \dots x_{n-k}) > 0 \\ BO(x_n | x_{n-1} \dots x_{n-k+1}), & \text{otherwise} \end{cases}$$

Backoff

Start with n-gram order of k but if the counts are 0 use $k - 1$:

$$BO(x_n|x_{n-1} \dots x_{n-k}) = \begin{cases} P(x_n|x_{n-1} \dots x_{n-k}), & \text{if } c(x_n \dots x_{n-k}) > 0 \\ BO(x_n|x_{n-1} \dots x_{n-k+1}), & \text{otherwise} \end{cases}$$

Is this a probability distribution?



$$P_{BO}(x_n | x_{n-1} \dots x_{n-k}) = \begin{cases} P^*(x_n | x_{n-1} \dots x_{n-k}), & \text{if } c(x_n \dots x_{n-k}) > 0 \\ \alpha^{x_{n-1} \dots x_{n-k}} P_{BO}(x_n | x_{n-1} \dots x_{n-k+1}), & \text{otherwise} \end{cases}$$

- with $\alpha^{x_{n-1} \dots x_{n-k}}$ the backoff weight
- computed so that probability distribution is respected (i.e., probs between 0 and 1 and sums to 1)
- See [Jurafsky and Martin, 2018]

Bibliography

- Chapter 3 from Jurafsky & Martin
- Chapter 6 from Eisentein
- Michael Collins' notes on LMs